# Heart Disease Risk Prediction

An End-to-End MLOps Pipeline

Machine Learning Operations (AIMLCZG523) - Assignment 01

BITS Pilani, Work Integrated Learning Programmes

Submitted by: Shivangi Nanda

BITS ID: 2025ae05562

Repository: https://github.com/shivanginanda7/mlops-heartdisease

November 2026

# 1. Introduction

The aim of this assignment is to build a classifier that predicts whether a patient has heart disease from routine clinical measurements, and to take it all the way to a monitored service running on Kubernetes. The machine learning part is deliberately small: the UCI Cleveland dataset has 303 patients and 13 features. Most of the work is in the engineering around the model: experiment tracking, packaging, automated testing, containerisation, deployment and monitoring.

The project is organised in four stages, shown in Figure 1. In development, the data is downloaded, cleaned and explored, and three model families are tuned and tracked in MLflow. Every push to GitHub runs a CI pipeline that lints the code, runs the unit tests, retrains the model and builds and tests the Docker image. The champion model is served by a FastAPI application inside a Docker image and deployed to a local Minikube cluster behind an NGINX Ingress. Prometheus scrapes the running pods and Grafana shows request, latency, error and prediction metrics, including a simple drift signal.

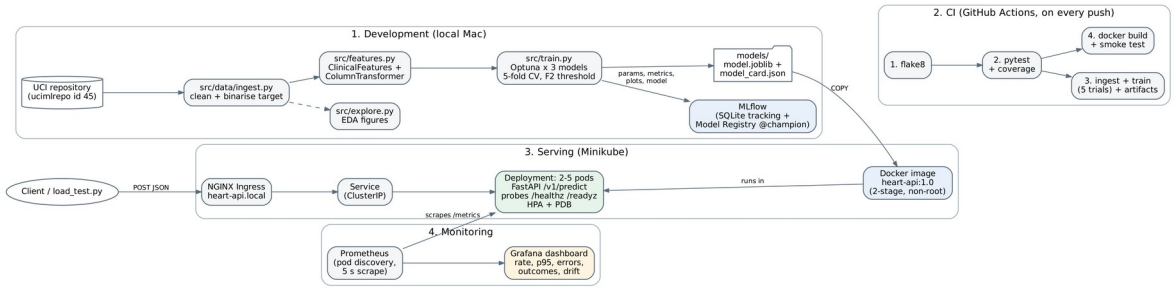



**Figure 1.** System architecture: development, CI, serving and monitoring.

One design decision runs through the whole project. In a screening setting, missing a patient who has the disease is worse than raising a false alarm, so the models are compared on recall-weighted metrics and the decision threshold is tuned for recall rather than left at 0.5.

# 2. Setup and Reproducibility

All code is in the GitHub repository listed on the title page. The README gives the full commands; the short version is below. The project uses Python 3.12. On macOS, XGBoost also needs the OpenMP runtime (brew install libomp).

- Clone the repository, create a virtual environment and run pip install -r requirements.txt.
- python -m src.data.ingest downloads and cleans the data; python -m src.explore produces the EDA figures.
- python -m src.train runs the tuning and writes the champion model to models/; python -m pytest runs the tests.
- uvicorn src.api.app:app --port 8000 starts the API locally; the Docker, Kubernetes and monitoring steps are in README sections 4 to 6.

Every step is a module run from the repository root, and all settings (paths, seed, column groups, number of folds and trials) live in src/settings.py, so a fresh clone reproduces the same results. The random seed is fixed at 2024 for the data split, the cross-validation folds and Optuna.

# 3. Data Acquisition and Exploratory Analysis

## 3.1 Acquisition and cleaning

src/data/ingest.py downloads the Cleveland subset through the ucimlrepo package (dataset id 45). If that call fails, it falls back to the original processed.cleveland.data file on the UCI site. The raw copy is saved to data/raw/ and the cleaned one to data/processed/, together with a small JSON report of what the cleaning changed.

The cleaning steps are:

- **Missing values:** the original file marks missing values with "?". All columns are converted to numbers and these become NaN. Only two columns have gaps: major_vessels (ca) has 4 and thal has 2. They are left as NaN on purpose and filled inside the model pipeline, so that the same logic runs at training and prediction time.
- **Target:** the original num column runs from 0 (no disease) to 4. It is binarised to has_disease = 1 for any value above 0. This gives 164 patients without disease and 139 with it (54% / 46%), so the classes are close to balanced.
- **Validation:** any categorical code outside the UCI data dictionary (for example a chest pain type other than 1 to 4) is set to missing rather than guessed. Duplicate rows are dropped. Neither occurred in this data, but the checks are covered by unit tests.
- **Readable names:** the short UCI names are renamed (cp to chest_pain, thalach to max_hr, oldpeak to st_depression and so on), and the same names are used in training, the API and the report.

## 3.2 Exploratory analysis

src/explore.py produces five figures and a text summary. Figures 2 to 6 show the main results.

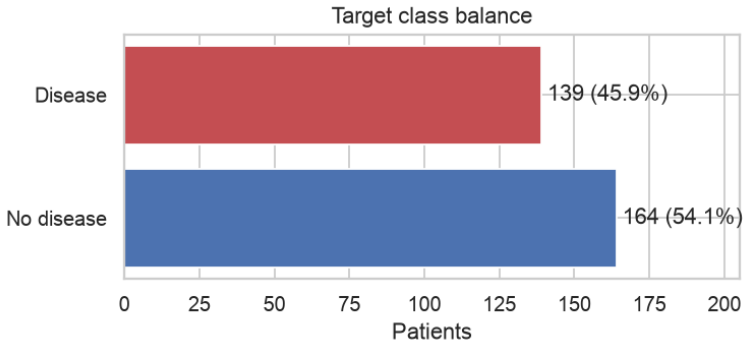



**Figure 2.** Class balance of the binarised target.

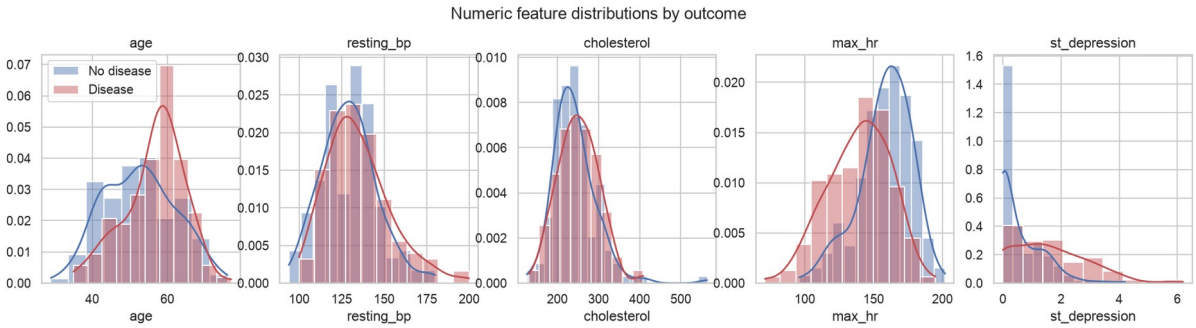



**Figure 3.** Distribution of the numeric features, split by outcome.

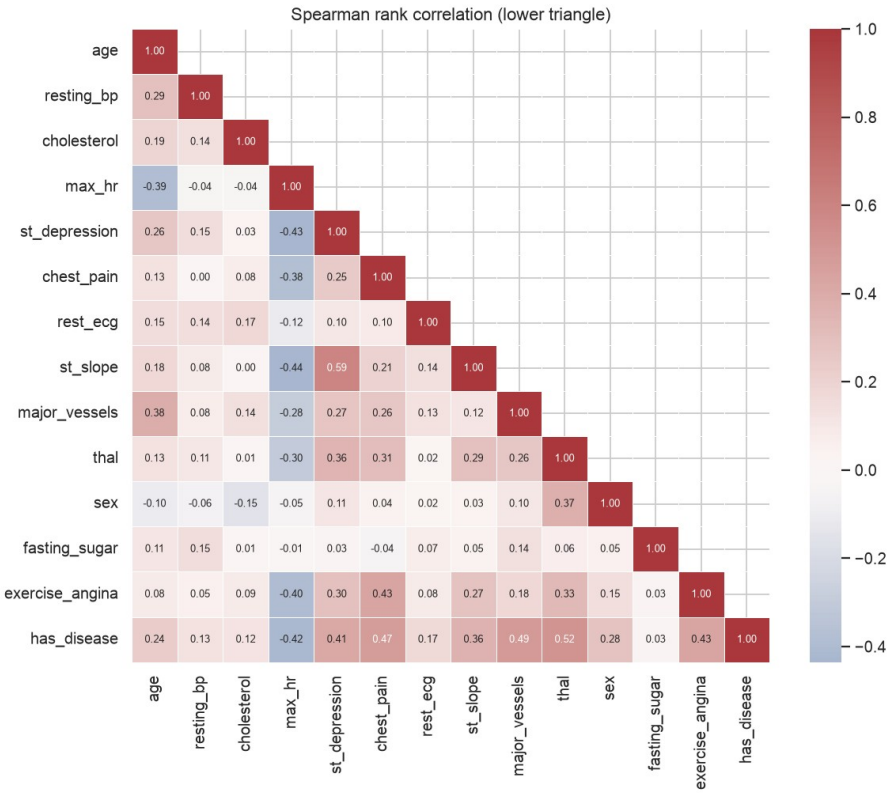



**Figure 4.** Spearman rank correlation between all features and the target.

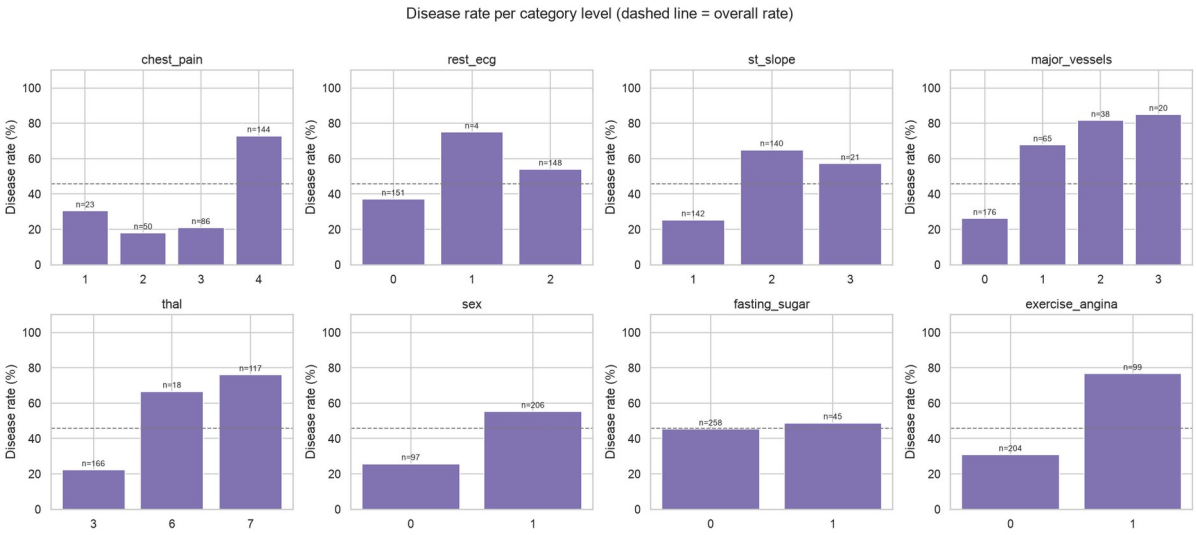



**Figure 5.** Disease rate for each level of the categorical features (n = patients in each group).

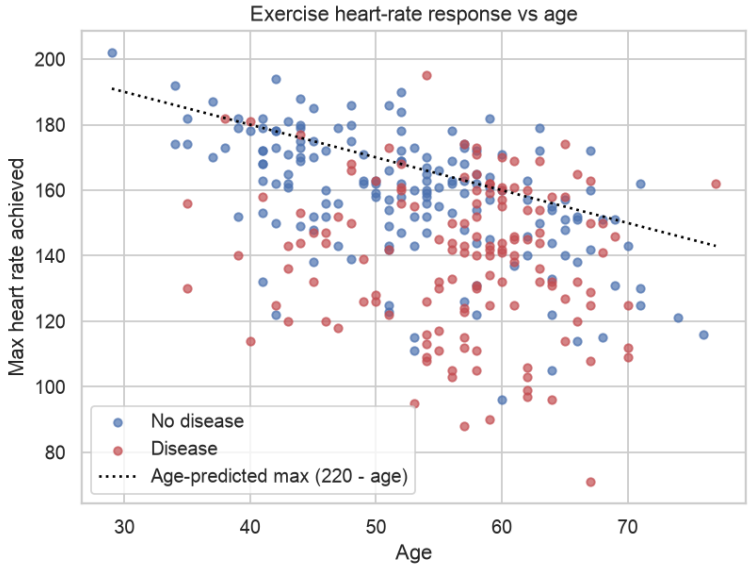



**Figure 6.** Maximum heart rate against age; the dotted line is the age-predicted maximum (220 - age).

The main observations from the EDA were:

- **Thal, major vessels and chest pain type are the strongest signals.** They have the highest Spearman correlations with the target (0.52, 0.49 and 0.47). Patients with a reversible defect (thal = 7) have a disease rate of about 76%, against about 22% for normal thal. The disease rate climbs steadily with the number of coloured vessels, from about 26% with none to about 85% with three.
- **Asymptomatic chest pain (type 4) is the high-risk group.** About 73% of these 144 patients have disease, compared with 18% to 30% for the three symptomatic types. This looks counter-intuitive at first but is a known feature of this dataset.
- **Exercise test results separate the classes well.** Patients with disease reach a lower maximum heart rate (correlation -0.42) and show more ST depression (0.41). Exercise-induced angina raises the disease rate from about 31% to about 77%. ST slope and ST depression are themselves correlated (0.59), so they carry partly the same information.
- **Sex matters, fasting sugar does not.** The disease rate is about 55% for men and 26% for women. Fasting blood sugar shows almost no difference (correlation 0.03).
- **Cholesterol and resting blood pressure** overlap heavily between the two groups (correlations 0.12 and 0.13) and carry little signal on their own.
- **Age and maximum heart rate are linked** (correlation -0.39). In Figure 6, many patients with disease fall well below their age-predicted maximum, while most patients without disease sit close to it. This motivated the pct_max_hr feature described in Section 4.
- **Small groups.** Some category levels are very small (only 4 patients have rest_ecg = 1, and 18 have thal = 6), so their disease rates are unreliable. This is one reason the one-hot encoder ignores unseen levels instead of failing.

# 4. Feature Engineering and Model Development

## 4.1 Preprocessing pipeline

All preprocessing lives in one scikit-learn Pipeline (src/features.py), and the fitted pipeline, including the model, is what gets saved and served. This means a single JSON patient in the

API goes through exactly the same steps as the training data, and nothing is fitted on the test set or on a full fold before splitting.

- **Numeric** (age, resting_bp, cholesterol, max_hr, st_depression): median imputation, then standard scaling.
- **Categorical** (chest_pain, rest_ecg, st_slope, major_vessels, thal): most-frequent imputation, then one-hot encoding with unknown categories ignored, so an unseen code at prediction time does not crash the service.
- **Binary** (sex, fasting_sugar, exercise_angina): most-frequent imputation, kept as 0/1.

### 4.2 Engineered features

Two features were added, each with a clinical reason:

- **pct_max_hr** = max_hr / (220 - age). This is the share of the age-predicted maximum heart rate the patient actually reached. Failing to reach it (chronotropic incompetence) is a known marker of coronary disease, and the ratio removes the age effect that raw max_hr carries.
- **exercise_ischaemia** = 1 when the patient has exercise-induced angina and an ST depression above 1 mm. It flags patients where the symptom and the ECG sign agree.

Rather than deciding up front whether these help, the choice was made a tuning parameter, so the search could switch them on or off. After encoding there are 27 input columns with the engineered features and 25 without.

### 4.3 Models and tuning

Three model families were compared: an elastic-net Logistic Regression as an interpretable baseline, an RBF-kernel SVM, and XGBoost. For the SVM, scikit-learn 1.9 deprecates probability=True, so the SVM is wrapped in CalibratedClassifierCV with Platt scaling to give usable probabilities.

The data is split 80/20 with stratification (242 training and 61 test patients). Each model gets 25 Optuna trials (TPE sampler), and every trial is scored by mean ROC-AUC over 5-fold stratified cross-validation on the training set only. ROC-AUC was used for tuning because it does not depend on a threshold.

### 4.4 Decision threshold and model selection

After tuning, each model's out-of-fold probabilities are used to choose a decision threshold. The threshold maximises the F2 score, which weights recall twice as heavily as precision. On its own, F2 tended to push the threshold to the bottom of the range and flag almost everyone, so a constraint was added: precision must stay at or above 0.65. Figure 7 shows how precision, recall and F2 move with the threshold for the champion model.

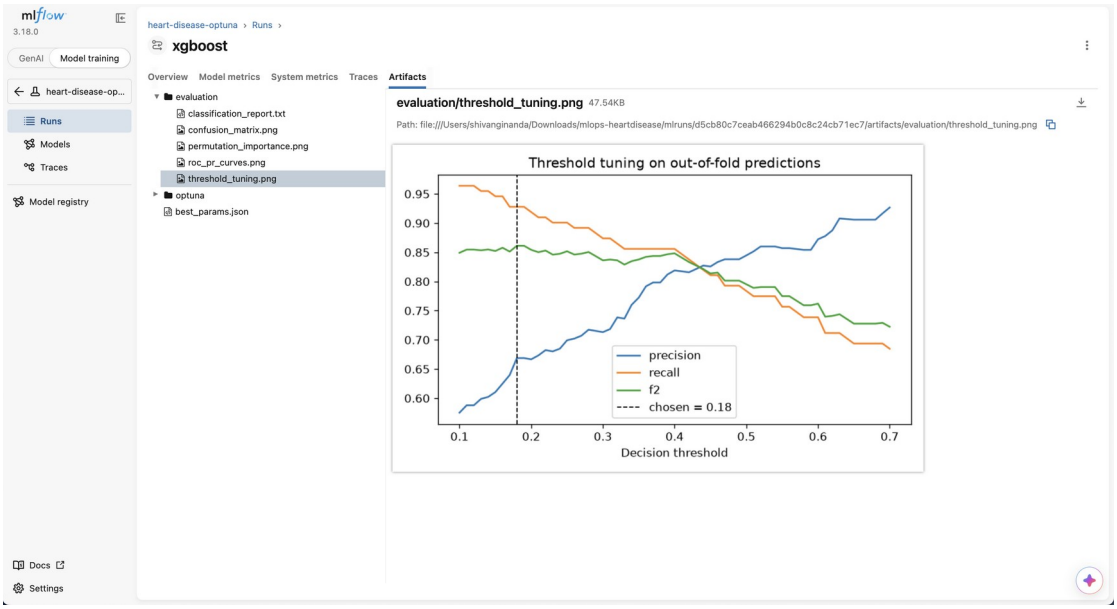



**Figure 7.** Threshold tuning for the XGBoost champion on out-of-fold predictions (from MLflow).

The champion is the model with the highest cross-validated F2, with ROC-AUC as a tie-breaker. The test set is used only for the final report, never for selection. Table 1 shows the results.

**Table 1.** Cross-validated results at each model's tuned threshold (mean over 5 folds).

| Model | ROC-AUC | F2 | Recall | Precision | Accuracy | Threshold |
|---|---|---|---|---|---|---|
| XGBoost (champion) | 0.916 ± 0.019 | 0.860 | 0.928 | 0.679 | 0.756 | 0.18 |
| SVM (RBF) | 0.910 ± 0.029 | 0.844 | 0.910 | 0.661 | 0.740 | 0.20 |
| Logistic Regression | 0.910 ± 0.028 | 0.836 | 0.883 | 0.702 | 0.768 | 0.30 |

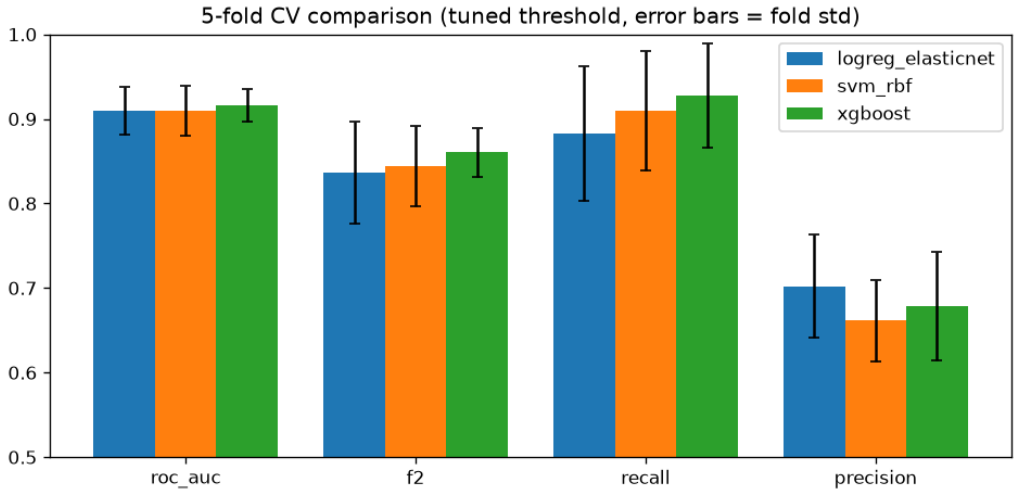



**Figure 8.** Cross-validated comparison of the three tuned models (error bars show the fold standard deviation).

XGBoost had the best F2 (0.860) and recall (0.928), and Optuna chose to keep the engineered features for it. The margin over the other two models is small, smaller than one fold standard

deviation, so the three models are close in practice. Logistic Regression has the best precision and accuracy and would be a reasonable choice if false alarms mattered more, or if interpretability were the priority.

Table 2 shows the same models on the 61-patient hold-out set, each at its own tuned threshold.

**Table 2.** Results on the hold-out test set (61 patients).

| Model | Test ROC-AUC | Test recall | Test F2 |
|---|---|---|---|
| XGBoost (champion) | 0.870 | 1.000 | 0.875 |
| SVM (RBF) | 0.933 | 0.964 | 0.882 |
| Logistic Regression | 0.927 | 0.929 | 0.861 |

The champion found every patient with disease in the test set (recall 1.0), which is what the low threshold was chosen for. Its test accuracy is 0.672, lower than in cross-validation; that is the expected cost of a threshold of 0.18, which trades more false alarms for fewer missed patients.

The test set also tells a slightly different story from cross-validation. On these 61 patients, Logistic Regression and the SVM rank patients better (test ROC-AUC 0.927 and 0.933) than XGBoost (0.870). The model was not switched on that basis: with 61 patients, a few cases move these numbers noticeably, and choosing a model by its test score would turn the test set into a second validation set. It does support the earlier point that the three models are close, and that the simpler Logistic Regression would be a defensible alternative.

## 5. Experiment Tracking

MLflow tracks every run, with a local SQLite backend (mlflow.db) and file-based artifact storage. The experiment is called heart-disease-optuna. Each model family gets a parent run, and each of its 25 Optuna trials is logged as a nested child run with its parameters and cross-validated ROC-AUC (Figure 9).

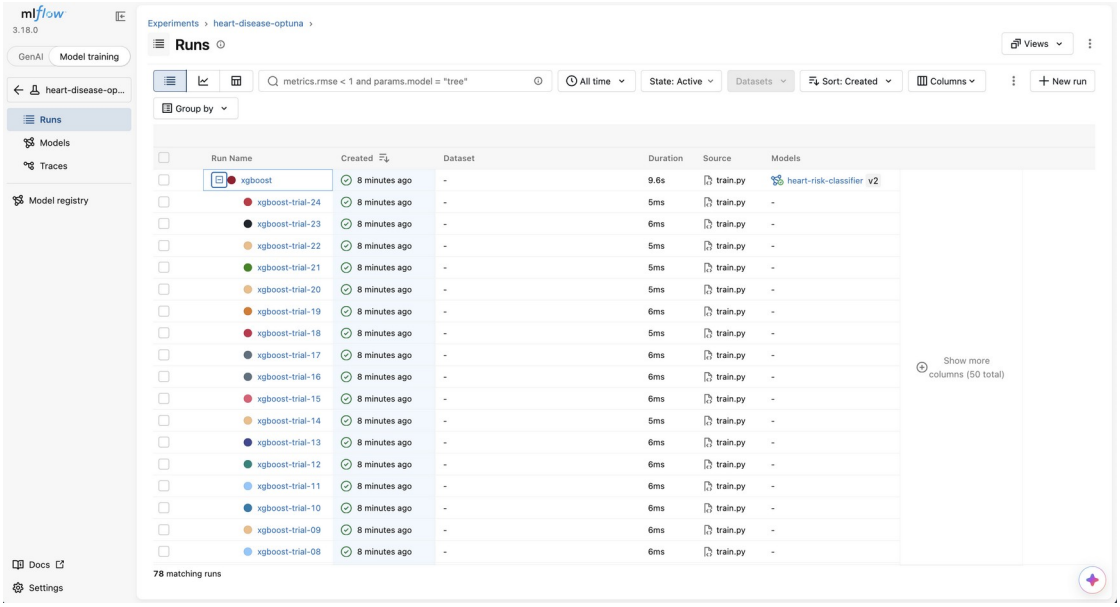



**Figure 9.** MLflow runs list: the XGBoost parent run expanded to show its Optuna trials.

Each parent run logs:

- **Parameters:** the best hyperparameters, whether the engineered features were used, the number of trials and folds, the F-beta value and the precision floor.
- **Metrics:** mean and standard deviation of accuracy, precision, recall, F1, F2 and ROC-AUC across folds, the chosen threshold, and the hold-out test metrics.
- **Artifacts:** ROC and precision-recall curves, a confusion matrix, the threshold tuning plot, permutation feature importance, a classification report, the full Optuna trials table, and the fitted pipeline with its input signature and an example.

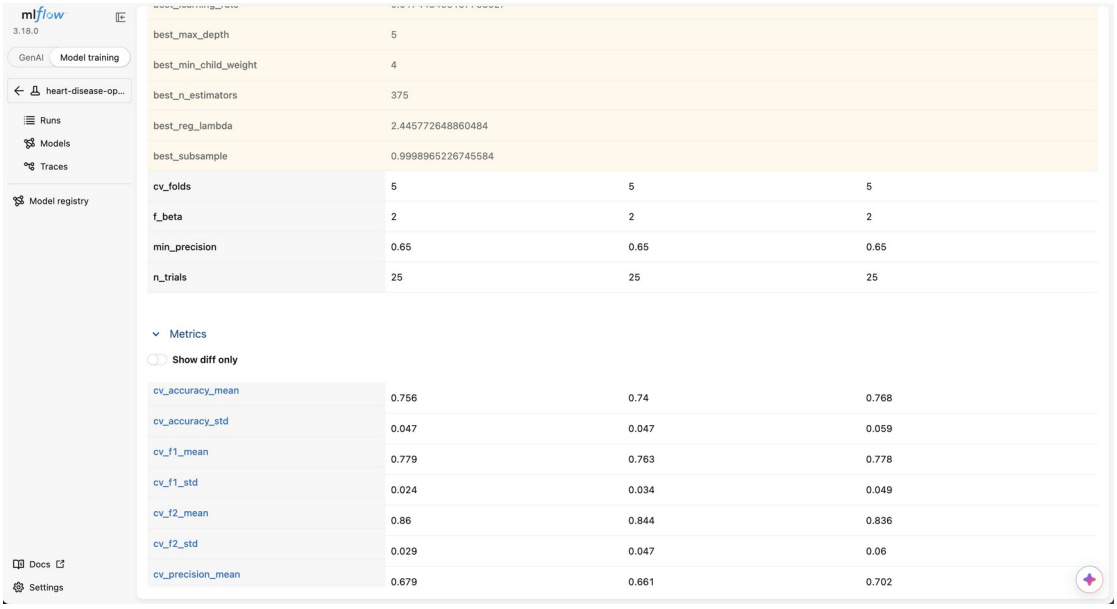



**Figure 10.** Comparing the three parent runs in MLflow.

The champion is registered in the MLflow Model Registry as heart-risk-classifier. Instead of the older stage labels, the current version is marked with the alias champion, and the chosen threshold is stored as a version tag (Figure 11). The training was run twice, which produced versions 1 and 2; the alias moved to version 2 automatically.

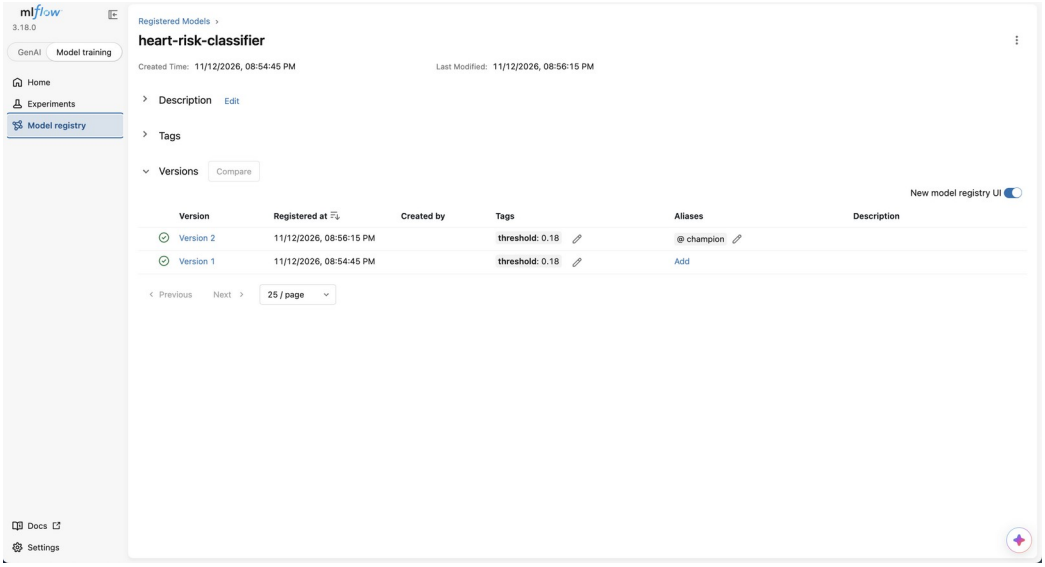



**Figure 11.** Model Registry: version 2 carries the champion alias and the threshold tag.

## 6. Model Packaging

The final artefact is the fitted Pipeline (engineered features, preprocessing and model in one object), saved in two forms:

- **models/model.joblib** with **models/model_card.json**, which records the model family, registry version, threshold, best parameters, feature list and metrics. The API reads both, so the threshold always travels with the model.
- **MLflow model format** in the run artifacts. MLflow 3.18 saves scikit-learn models with skops by default, and skops refuses to load the custom ClinicalFeatures class. The model is therefore logged with cloudpickle and the src/ folder is included as code, so it can be loaded elsewhere.

Two requirements files are used. requirements.txt covers development and CI. requirements-serve.txt holds only the ten packages the API needs, with exact versions taken from the environment that trained the model. A pickled scikit-learn model can fail to load under a different version, so pinning the serving image to the training versions avoids that.

## 7. Model Serving API

The model is served by a FastAPI application (src/api/app.py). The prediction logic is shared with a command-line tool (src/predict.py) through src/inference.py, so both apply the same threshold and return the same fields.

**Table 3.** API endpoints.

| Endpoint | Purpose |
|---|---|
| POST /v1/predict | One patient as JSON; returns prediction, label, disease probability, confidence, risk band, threshold and model version |
| GET /v1/model | Model card of the served model |
| GET /healthz | Liveness: the process is running |
| GET /readyz | Readiness: the model is loaded (503 until it is) |
| GET /metrics | Prometheus metrics |

Input is validated with a Pydantic schema before it reaches the model. Each field has a valid range or a list of allowed codes from the UCI data dictionary, unknown fields are rejected, and only major_vessels and thal may be null, since the pipeline imputes them. Invalid input returns HTTP 422 with a clear message (Figure 12).

Confidence is defined relative to the decision threshold rather than as the raw probability: it is 0 when the probability sits exactly on the threshold and rises to 1 as the probability approaches 0 or 1. With a threshold of 0.18, a raw probability of 0.2 would otherwise be reported as "disease with 20% confidence", which is misleading.

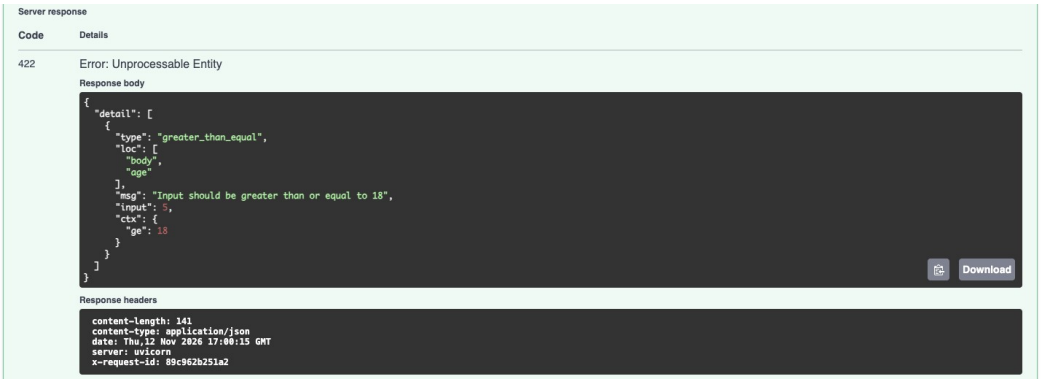



**Figure 12.** Validation: an age of 5 is rejected with HTTP 422.

Every request writes one JSON log line with a request id, path, status and latency. For predictions, the log holds a short SHA-256 hash of the input instead of the patient's values, so no clinical data ends up in the logs (Figure 13).

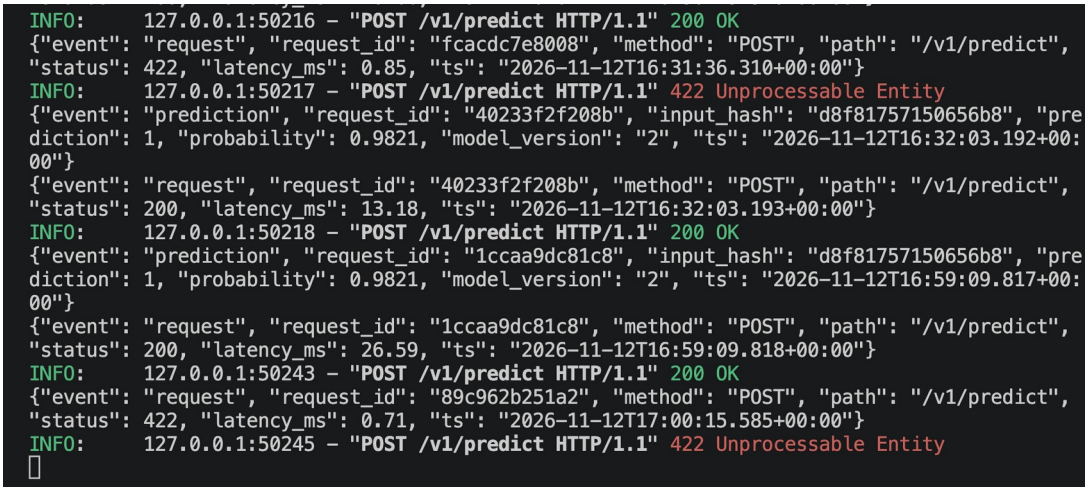



**Figure 13.** Structured JSON request and prediction logs from the API.

## 8. CI/CD Pipeline and Automated Testing

The test suite has 35 pytest tests covering cleaning (missing values, binary target, invalid codes, duplicates), the preprocessing pipeline (output shape, no NaNs after imputation, unseen categories, engineered values), each model's fit and predict, an end-to-end training run with a temporary MLflow store, and the API through FastAPI's TestClient (valid predictions, threshold logic, rejected inputs, readiness without a model, metrics). Line coverage is 86% (Figure 14).

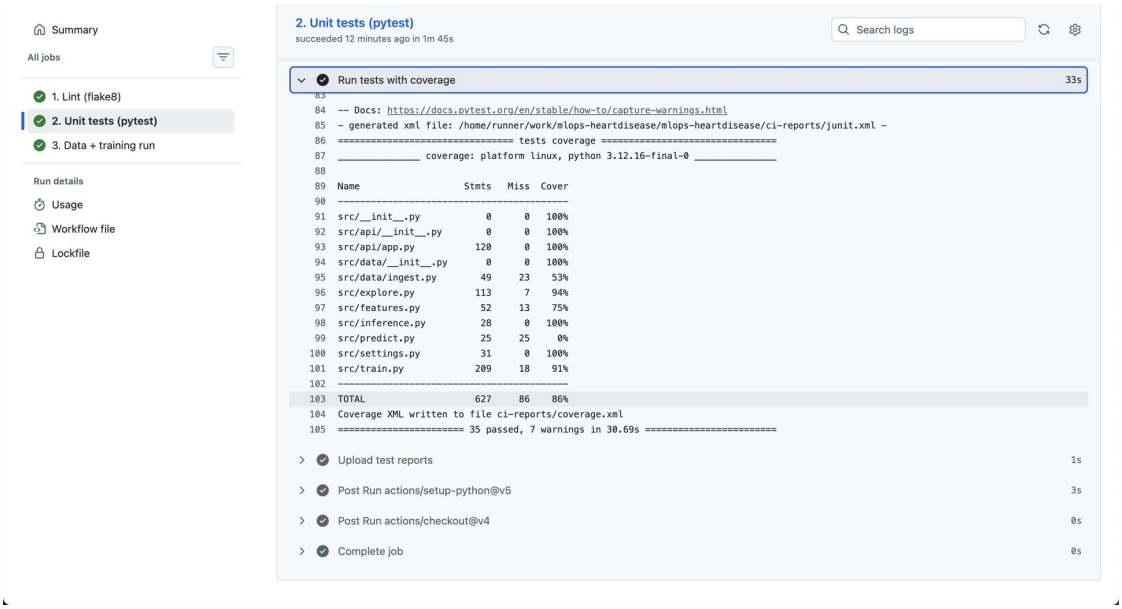



**Figure 14.** Test job in GitHub Actions: 35 tests passed with 86% coverage.

The GitHub Actions workflow (.github/workflows/ci.yml) runs on every push to main and has four jobs:

- **1. Lint:** flake8; any error is shown as an annotation on the exact file and line.
- **2. Unit tests:** pytest with coverage; the JUnit and coverage reports are uploaded as artifacts.
- **3. Data and training run:** downloads and cleans the data, makes the EDA figures, trains with a short 5-trial search to keep CI fast, smoke-tests the saved model, writes a results table to the run summary, and uploads the model, model card, metrics and plots.
- **4. Docker build and smoke test:** builds the image, waits for the container's health check to report healthy, sends a sample patient to /v1/predict and checks that invalid input returns 422.

Jobs 3 and 4 only start if the tests pass, and the tests only start if lint passes. To confirm that the pipeline fails as required, a test was broken on purpose: the run stopped at the test job with a clear assertion message and training never started (Figures 16 and 17). The test was then fixed and the next run was green.

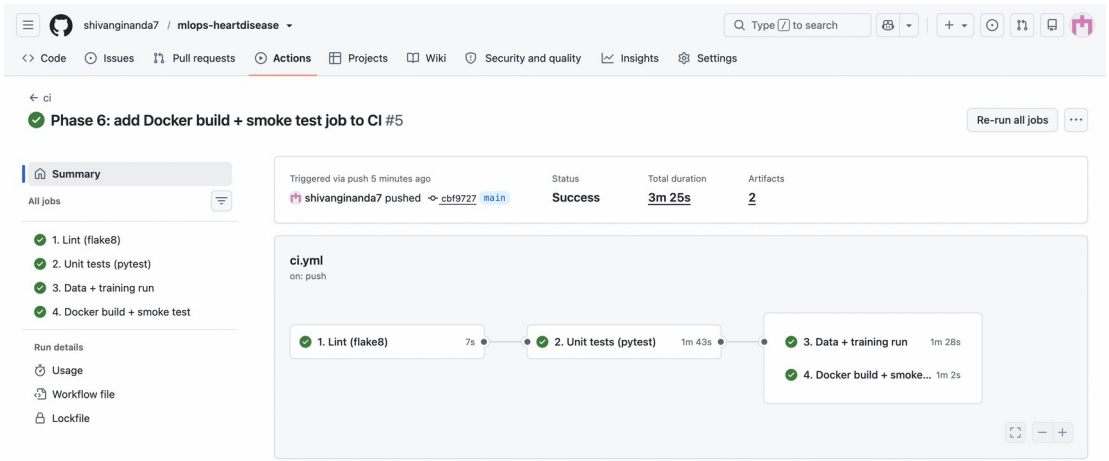



**Figure 15.** A green CI run with all four jobs: lint, tests, training and the Docker build and smoke test.

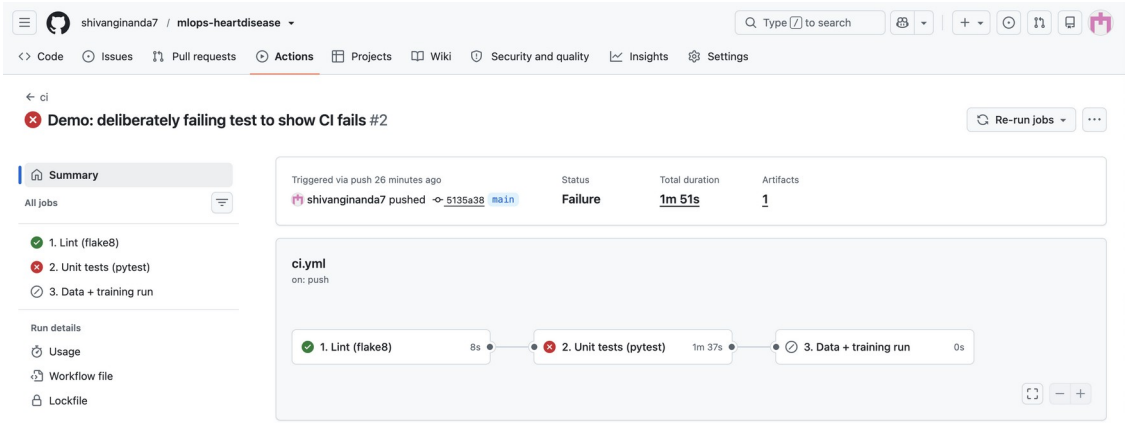



**Figure 16.** The deliberately broken test stops the pipeline at the test job; training is skipped.

```
142  ========================== short test summary info ==========================
143  FAILED tests/test_features.py::test_preprocessor_output_shape_and_no_nans - assert (40, 27) == (40, 28)
144
145    At index 1 diff: 27 != 28
146
147    Full diff: (-: missing in left side, +: extra in left side)
148      (
149          40,
150      -     28,
151      ?      ^
152      +     27,
153      ?      ^
154      )
155  ================== 1 failed, 34 passed, 7 warnings in 31.32s ===================
156  Error: Process completed with exit code 1.
```

**Figure 17.** The failing assertion in the job log, with the process exiting with code 1.

# 9. Containerisation

The Dockerfile uses a two-stage build on python:3.12-slim. The first stage installs requirements-serve.txt; the second copies only the installed packages, the src/ code and the model files. Training libraries (MLflow, Optuna, Jupyter), the data and the tests are left out, and a .dockerignore keeps the build context small. The container runs as a non-root user, and the libgomp1 runtime is added because XGBoost needs it on Linux.

The HEALTHCHECK calls /readyz rather than just checking that the process is up, so the container only reports healthy once the model is loaded. Since the slim image has no curl, the check uses Python's standard library.

```
((.venv) ) (base) shivanginanda@Mac mlops-heartdisease % docker build -t heart-api:1.0 .
docker image ls heart-api
 => [builder 4/4] RUN pip install --no-cache-dir --prefix=/install -r requirements-s  84.3s
 => [stage-1 3/7] RUN useradd --create-home --uid 10001 appuser                        0.1s
 => [stage-1 4/7] WORKDIR /app                                                         0.0s
 => [stage-1 5/7] COPY --from=builder /install /usr/local                              0.9s
 => [stage-1 6/7] COPY src/ ./src/                                                     0.0s
 => [stage-1 7/7] COPY models/model.joblib models/model_card.json ./models/            0.0s
 => exporting to image                                                                11.5s
 => => exporting layers                                                                9.0s
 => => exporting manifest sha256:5969e850c48d5ac49ced4d8edd0a0bd55d0168d9080e2ef0170d  0.0s
 => => exporting config sha256:bfca496a2820f389c2e3c9897ef0d7637773d8d0b106a24ab2d3d6  0.0s
 => => exporting attestation manifest sha256:0ef0cf0d2dd8df813a70a54b5ae6925836010e9a  0.0s
 => => exporting manifest list sha256:1f3305f81073d2146b5b723157885a91bbe9aab2610ef83  0.0s
 => => naming to docker.io/library/heart-api:1.0                                       0.0s
 => => unpacking to docker.io/library/heart-api:1.0                                    2.5s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/dbad94lnrd8ms
ty2d9vrmmj6g

IMAGE           ID             DISK USAGE   CONTENT SIZE   EXTRA
heart-api:1.0   1f3305f81073       1.28GB          417MB
((.venv) ) (base) shivanginanda@Mac mlops-heartdisease %
```

**Figure 18.** Building the image; the final image is 417 MB of content.

```
((.venv) ) (base) shivanginanda@Mac mlops-heartdisease % docker run -d --name heart-api -p 800
0:8000 heart-api:1.0
sleep 20
docker ps
cdf735398a548d3a900bcc7818420c526f138d3865076e0893823230f27f3c09
CONTAINER ID   IMAGE           COMMAND                  CREATED          STATUS
     PORTS                                       NAMES
cdf735398a54   heart-api:1.0   "uvicorn src.api.app…"   20 seconds ago   Up 20 seconds (health
y)   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp   heart-api
((.venv) ) (base) shivanginanda@Mac mlops-heartdisease %
```

**Figure 19.** The container running and reported healthy by Docker.

```
((.venv) ) (base) shivanginanda@Mac mlops-heartdisease % curl http://127.0.0.1:8000/readyz
echo
curl -X POST http://127.0.0.1:8000/v1/predict -H "Content-Type: application/json" -d @sample_p
atient.json
echo
docker logs heart-api
{"status":"ready","model_version":"2"}
{"request_id":"7daa69400494","prediction":1,"label":"disease","probability_disease":0.9821,"co
nfidence":0.9782,"risk_band":"high","threshold":0.18,"model_version":"2","model_family":"xgboo
st"}
INFO:     Started server process [1]
INFO:     Waiting for application startup.
{"event": "model_loaded", "model_dir": "/app/models", "version": "2", "family": "xgboost", "th
reshold": 0.18, "ts": "2026-11-14T09:00:59.674+00:00"}
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
{"event": "request", "request_id": "852072adc5e0", "method": "GET", "path": "/readyz", "status
": 200, "latency_ms": 0.64, "ts": "2026-11-14T09:01:03.870+00:00"}
INFO:     127.0.0.1:42742 - "GET /readyz HTTP/1.1" 200 OK
{"event": "request", "request_id": "a89685d2b83a", "method": "GET", "path": "/readyz", "status
": 200, "latency_ms": 0.26, "ts": "2026-11-14T09:01:19.024+00:00"}
```

**Figure 20.** Readiness check and a prediction served from inside the container, with its startup logs.

## 10. Production Deployment on Kubernetes

The API is deployed to a local Minikube cluster. The image is loaded straight into Minikube with minikube image load, and the manifests in k8s/ are applied in numbered order:

- **Namespace** heart-risk, keeping the application separate.
- **Deployment** with 2 replicas and a rolling update that starts a new pod before removing an old one. A startup probe gives the model time to load, a readiness probe on /readyz only sends traffic to pods with the model loaded, and a liveness probe on /healthz restarts a pod that stops responding. Each pod requests 100m CPU and 256 MiB and is limited to 500m and 512 MiB. Pods run as non-root with all Linux capabilities dropped, and carry annotations so Prometheus can find them.
- **Service** (ClusterIP) and an **NGINX Ingress** on host heart-api.local, which is how the API is exposed outside the cluster.
- **HorizontalPodAutoscaler** scaling between 2 and 5 pods at 70% average CPU, and a **PodDisruptionBudget** keeping at least one pod up during maintenance.

```
((.venv) ) (base) shivanginanda@Mac mlops-heartdisease % kubectl apply -f k8s/
kubectl -n heart-risk rollout status deployment/heart-api
kubectl -n heart-risk get pods,svc,ingress,hpa
namespace/heart-risk created
deployment.apps/heart-api created
service/heart-api created
ingress.networking.k8s.io/heart-api created
horizontalpodautoscaler.autoscaling/heart-api created
poddisruptionbudget.policy/heart-api created
Waiting for deployment "heart-api" rollout to finish: 0 of 2 updated replicas are available...
Waiting for deployment "heart-api" rollout to finish: 1 of 2 updated replicas are available...
deployment "heart-api" successfully rolled out
NAME                             READY   STATUS    RESTARTS   AGE
pod/heart-api-676bbc9d55-hg7sv   1/1     Running   0          4s
pod/heart-api-676bbc9d55-kmnkm   1/1     Running   0          4s

NAME                 TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE
service/heart-api    ClusterIP   10.103.16.82   <none>        80/TCP    4s

NAME                                    CLASS   HOSTS              ADDRESS        PORTS   AGE
ingress.networking.k8s.io/heart-api     nginx   heart-api.local    192.168.49.2   80      4s

NAME                                              REFERENCE               TARGETS              MI
NPODS   MAXPODS   REPLICAS   AGE
horizontalpodautoscaler.autoscaling/heart-api     Deployment/heart-api    cpu: <unknown>/70%   2
        5         2          4s
```

**Figure 21.** Deployment rolled out: two running pods, the service, the ingress and the autoscaler.

```
((.venv) ) (base) shivanginanda@Mac mlops-heartdisease % curl -H "Host: heart-api.local" http:
//127.0.0.1/readyz
echo
curl -X POST -H "Host: heart-api.local" -H "Content-Type: application/json" -d @sample_patient
.json http://127.0.0.1/v1/predict
{"status":"ready","model_version":"2"}
{"request_id":"c6f6fb7159bee93f5b29dd1c670952db","prediction":1,"label":"disease","probability
_disease":0.9821,"confidence":0.9782,"risk_band":"high","threshold":0.18,"model_version":"2","
model_family":"xgboost"}%
```

**Figure 22.** A prediction served through the Ingress.

The request id in Figure 22 is 32 characters long because the NGINX Ingress adds its own X-Request-ID header, which the API reuses. A single id therefore follows a request from the ingress to the pod logs. To test self-healing, one pod was deleted; Kubernetes created a replacement within two seconds (Figure 23).

```
((.venv) ) (base) shivanginanda@Mac mlops-heartdisease % kubectl -n heart-risk delete pod $(ku
bectl -n heart-risk get pods -o name | head -1 | cut -d/ -f2)
kubectl -n heart-risk get pods
pod "heart-api-676bbc9d55-hg7sv" deleted from heart-risk namespace
NAME                         READY   STATUS    RESTARTS   AGE
heart-api-676bbc9d55-kmnkm   1/1     Running   0          6m8s
heart-api-676bbc9d55-tzh4h   0/1     Running   0          2s
```

**Figure 23.** Self-healing: a deleted pod is replaced immediately.

## 11. Monitoring and Logging

Prometheus and Grafana run in a separate monitoring namespace (monitoring/). Prometheus uses Kubernetes pod discovery limited to the heart-risk namespace and keeps only pods with the prometheus.io/scrape annotation, scraping every 5 seconds. Replacement pods, such as the one from the self-healing test, are picked up without any change (Figure 24).

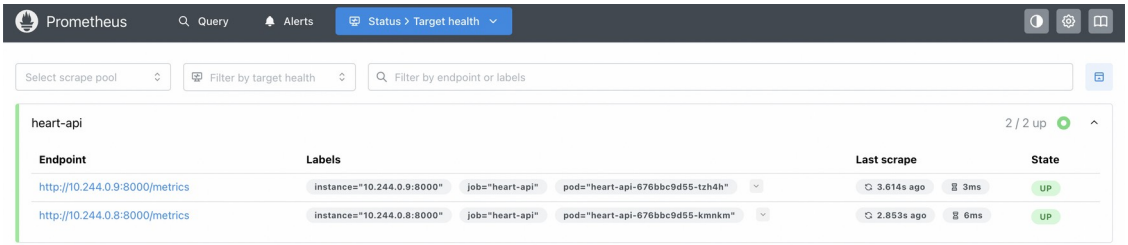



**Figure 24.** Prometheus discovers and scrapes both API pods.

Grafana is configured entirely from files: the Prometheus data source and the dashboard are loaded at startup from ConfigMaps, and the dashboard JSON is kept in the repository. It has eight panels: pods up, predictions served, error rate, request rate by status, p50 and p95 latency, predictions by outcome, and a drift signal shown both as a current value and over time.

The API exports its own metrics: request counts by path and status, a latency histogram, predictions by outcome, a histogram of predicted probabilities, and a gauge with the mean predicted probability over each pod's last 100 predictions. That last gauge is the drift signal. If the incoming patients change, it moves before any true labels are available. It turns orange at 0.6 and red at 0.75.

scripts/load_test.py generates traffic by sampling real patients from the cleaned data, with about 5% deliberately invalid requests. Under normal traffic (Figure 25), the dashboard showed about 8 requests per second, a p95 latency of about 25 ms, an error rate of about 4% (the deliberate bad requests) and a drift signal of about 0.45 to 0.5. Running the script with --drift sends an older population with more exercise-induced ischaemia, and the drift signal rose to 0.775, into the red zone (Figure 26). The share of disease predictions moved from about two thirds to 76% at the same time. In a real deployment this would be the trigger to look at the incoming data and consider retraining.

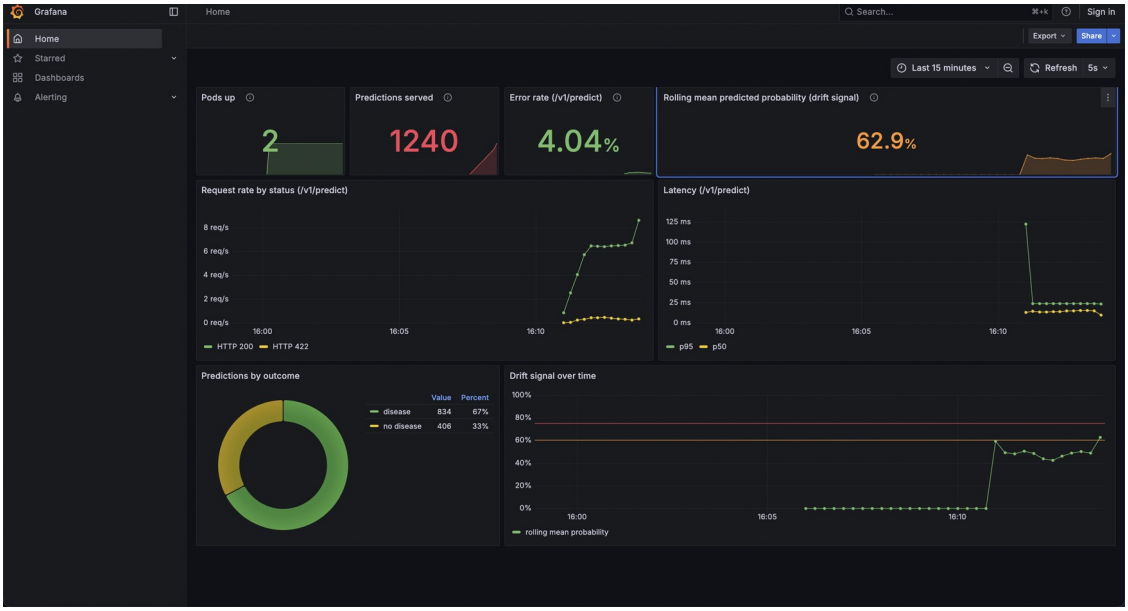



**Figure 25.** Grafana dashboard: normal traffic, with the start of the drifted run visible on the right of the drift panels.

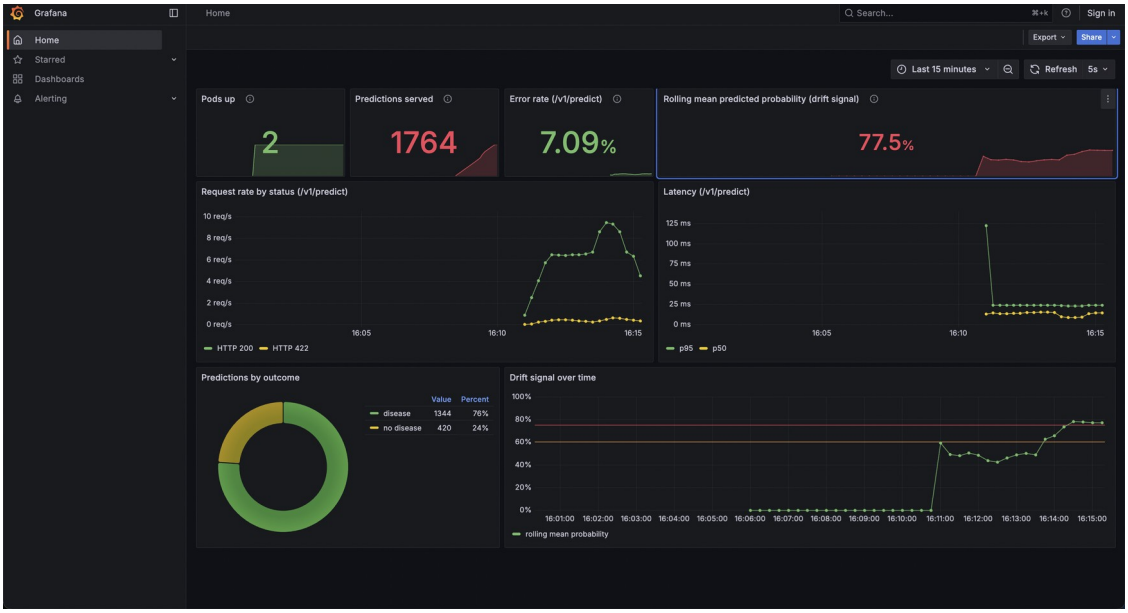



**Figure 26.** After the drifted run: the drift signal reaches 77.5% and turns red.

## 12. Limitations and Future Work

- **Small, single-site data.** 303 patients from one hospital is a small and dated sample, and the model has not been checked on other populations, for example the Hungarian or Swiss subsets of the same UCI collection.
- **Small test set.** With 61 test patients, a few cases move the test metrics noticeably. Repeated cross-validation or a bootstrap confidence interval would give a fuller picture.
- **Drift signal without labels.** The rolling mean probability detects changes in the inputs the model sees, not changes in accuracy. A production system would also log true outcomes once known and track recall over time, and could add per-feature drift tests.
- **Local infrastructure.** MLflow uses a local SQLite file, and the image is loaded into Minikube rather than pushed to a registry. Next steps would be a shared MLflow server, pushing images to a registry from CI, and deploying to a managed cluster with alerting rules in Prometheus.

## 13. Deliverables

| Item | Location |
|---|---|
| Code repository | https://github.com/shivanginanda7/mlops-heartdisease |
| Setup and access instructions | README.md |
| Data download and cleaning | src/data/ingest.py, data/processed/ |
| EDA, training and inference | src/explore.py, src/train.py, src/predict.py |
| Unit tests | tests/ |
| CI workflow | .github/workflows/ci.yml |
| Dockerfile and requirements | Dockerfile, requirements.txt, requirements-serve.txt |
| Kubernetes manifests | k8s/ |
| Monitoring | monitoring/, scripts/load_test.py |

| Item | Location |
| --- | --- |
| Screenshots | screenshots/ |